Debugging

CSC 230 : C and Software Tools

NC State Department of Computer Science

Topics For Today

- Limiting GCC output
- Meet assert()
- GCC code instrumentation (some new friends)
- Code coverage
- Meet valgrind (your new best friend)
- Meet cppcheck

Limiting GCC Messages

- GCC can generate a lot of messages.
  - Usually, you want to start with the first one.
  - You can scroll back, but that's inconvenient.

```
static int unused_A;
static int unused_B;

int main()
{
  undefined_X = 5;
  undefined_Y = 10;

  return EXIT_SUCCESS;
}
```

We'll cause warnings.

We'll cause errors.

Limiting GCC Messages

- GCC tries to compile as much as it can.
  - Reporting errors and warnings as it goes.

```
$ gcc -Wall -std=c99 -g program.c
program.c: In function ‘main’:
program.c:10:3: error: ‘undefined_X’ undeclared (first use in this function)
   undefined_X = 5;
   ^
program.c:11:3: error: ‘undefined_Y’ undeclared (first use in this function)
   undefined_Y = 10;
   ^
program.c: At top level:
program.c:5:12: warning: ‘unused_A’ defined but not used [-Wunused-variable]
 static int unused_A;
            ^
program.c:6:12: warning: ‘unused_B’ defined but not used [-Wunused-variable]
 static int unused_B;
```

Limiting GCC Messages

- The -fmax-errors=n option will limit errors

```
$ gcc -fmax-errors=1 -Wall -std=c99 -g program.c
program.c: In function ‘main’:
program.c:10:3: error: ‘undefined_X’ undeclared (first use in this function)
   undefined_X = 5;
   ^
compilation terminated due to -fmax-errors=1.
```

Limiting GCC Messages

- But, this option doesn't limit warnings

```
$ gcc -fmax-errors=3 -Wall -std=c99 -g program.c
program.c: In function 'main':
program.c:10:3: error: 'undefined_X' undeclared (first use in this function)
   undefined_X = 5;
   ^
program.c:11:3: error: 'undefined_Y' undeclared (first use in this function)
   undefined_Y = 10;
   ^
program.c: At top level:
program.c:5:12: warning: 'unused_A' defined but not used
 static int unused_A;
            ^
program.c:6:12: warning: 'unused_B' defined but not used [-Wunused-variable]
 static int unused_B;
            ^
```

All warnings still reported.

Limiting GCC Messages

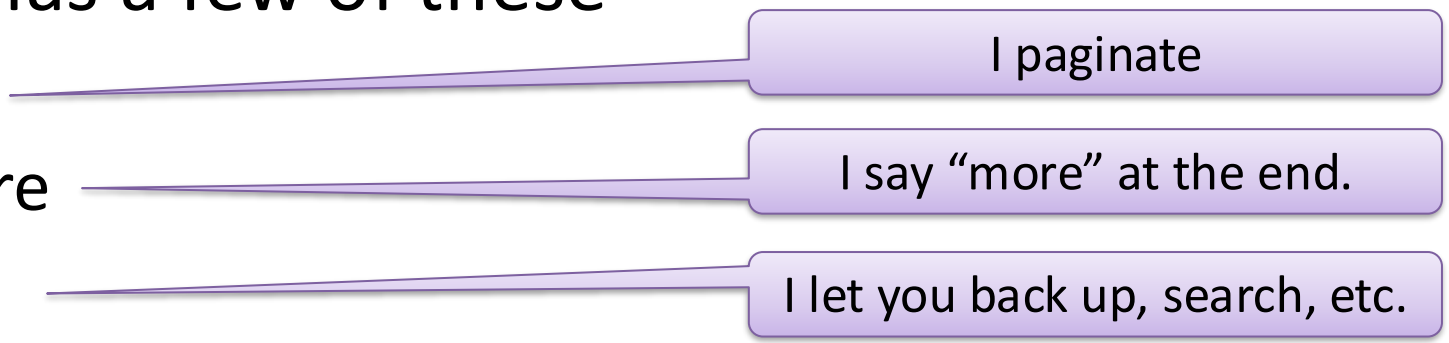
- You can promote warnings to errors: -Werror

```
$ gcc -Werror -fmax-errors=3 -Wall -std=c99 -g program.c
program.c: In function ‘main’:
program.c:10:3: error: ‘undefined_X’ undeclared (first use in this function)
   undefined_X = 5;
   ^
program.c:11:3: error: ‘undefined_Y’ undeclared (first use in this function)
   undefined_Y = 10;
   ^
program.c: At top level:
program.c:5:12: error: ‘unused_A’ defined but not used
 static int unused_A;
            ^
compilation terminated due to -fmax-errors=3.
cc1: all warnings being treated as errors
```

Now we stop after 3.

Paginated Output

- You can send the output of a program through a paginator.
- Unix has a few of these
  - pg
  - more
  - less
- These let you page through program output
- … maybe even backing up or searching through it.



Paginated Compiler Output

- To use these paginator programs, you can pipe standard output to them.

```
$ ./some-program | more
```

I'm a pipe.

Output of the first program.

… becomes input of the secondone.

- This doesn't work with compiler error / warning messages.
  - Those go to standard error, not standard output.

```
$ gcc -Wall -std=c99 program.c | more
```

Output still scrolls by.

Paginated Compiler Output

- Remember, we can redirect stderr … to the same place as stdout.

```
$ gcc -Wall -std=c99 program.c 2>&1 | less
```

OK. Now this gets paginated.

Also works for make.

- The same technique will let you see just the start of compiler output.

```
$ gcc -Wall -std=c99 program.c 2>&1 | head
```

I discard all but the first 10 lines.

assert() in C

- We can build tests right into our source code
- Like Java, C has support for assert()
  - It's provided via the preprocessor:
    #include <assert.h>
- Usage:

  `assert(` *some-condition-you-expect-to-be-true* `);`
- We can use this to build sanity-checks into our code

```
#include <assert.h>

int f( int a, int b )
{
   assert( a != 0 && b > 0 );
   …;
}
```

Planning for Failure

- You can use assert() to comment on your assumptions as write.

```
assert( i < length );
…
assert( x >= 0 && y >= 0 );
…
assert( ptr != NULL );
```

- But, these are comments with teeth!
- They will terminate your program if they are violated.

```
Assertion failed: (j >= 0 && j + 1 < len),\
function sortList, file bubble3b.c, line 50.
```

Assert as Executable Comments

- We had a good example of assert in our resizable array

```
if ( len >= capacity ) {
  capacity *= 2;
  list = (int *)realloc( list, capacity * sizeof( int ) );
}

assert( len < capacity );

list[ len++ ] = val;
```

Grow the array if needed.

It really, really better be big enough now

… because I'm about to use that next slot.

Using and Not Using assert()

- With assert(), we can selectively include bounds checking or other sanity checks in our code.
- Checking assertions cause additional run-time overhead,
  - But, we can use conditional compilation to disable assertions in production code:

```
gcc -DNDEBUG -std=c99 -Wall …
```

Instrumentation via GCC

- GCC can add extra instrumentation code as it compiles your program.
  - To watch what your program is doing as it runs
  - And to help detect some types of errors
- These options (mostly) start with “-f”

Option	Description
-fsanitize=address	A large set of pointer checks.
-fsanitize=undefined	A large set of checks for undefined behavior

- Some newer features don’t work on the common platform (older compiler)

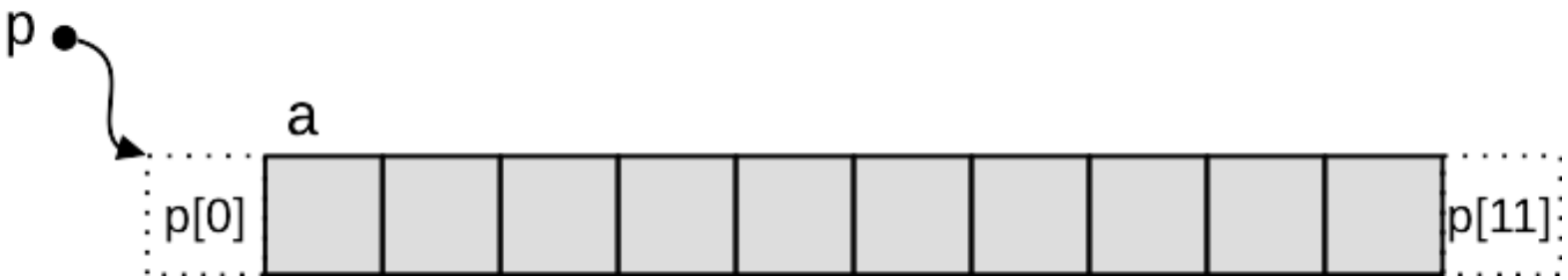
Stack Buffer Overflow

- Access off the ends of a stack variable?

```
int a[ 10 ] = { 1, 2, 3 };
int *p = a - 1;
printf( "%d\n", p[ 0 ] );
printf( "%d\n", p[ 11 ] );
```

Backing up one element … Bad Idea.

Both out of bounds



Stack Buffer Overflow

- Access off the ends of a stack variable?

```
int a[ 10 ] = { 1, 2, 3 };
int *p = a - 1;
printf( "%d\n", p[ 0 ] );
printf( "%d\n", p[ 11 ] );
```

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
=================================================================
==509074==ERROR: AddressSanitizer: stack-buffer-underflow ...
READ of size 4 at 0x7ffd5fe7052c thread T0
    #0 0x563a6f5e15b0 in function program.c:9
...
```

This doesn't work on the common platform. ☹ But it does on our VM. ☺

More output I'm not showing.

Heap Buffer Overflow

- Access off the ends of a heap block?

```
int *a = (int *) malloc( sizeof( int ) * 10 );
int *p = a - 1;
p[ 0 ] = 100;
p[ 11 ] = 200;
```

Backing up one element.



Heap Buffer Overflow

- Access off the ends of a heap block?

```
int *p = (int *) malloc( sizeof( int ) * 10 );
int *p = a - 1;
p[ 0 ] = 100;
p[ 11 ] = 200;
```

Backing up one element.

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
=================================================================
==508970==ERROR: AddressSanitizer: heap-buffer-overflow ...
WRITE of size 4 at 0x60400000000c thread T0
    #0 0x55fcd87a41fe in main program.c:9
```

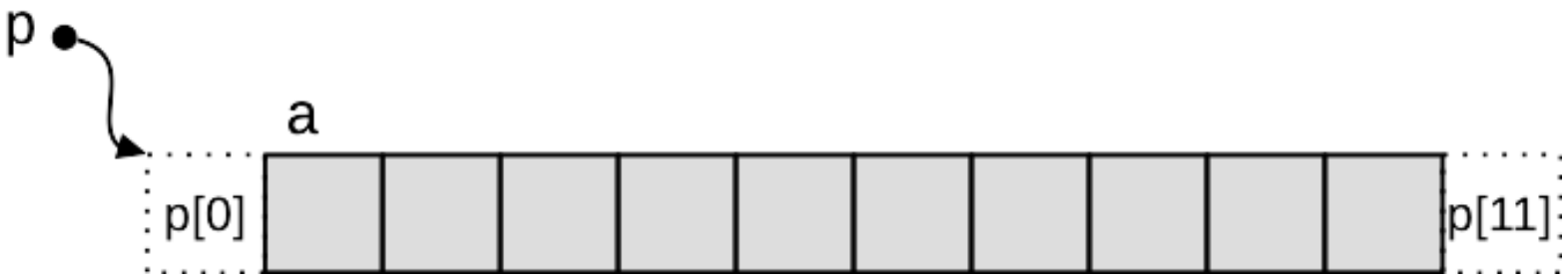
Static Buffer Overflow

- Access off the ends of a static variable?

```
int a[ 10 ];

void function()
{
  int *p = a - 1;
  printf( "%d\n", p[ 0 ] );
  printf( "%d\n", p[ 11 ] );
}
```

Backing up one element.



Static Buffer Overflow

- Access off the ends of a static variable?

```
int a[ 10 ];

void function()
{
  int *p = a - 1;
  printf( "%d\n", p[ 0 ] );
  printf( "%d\n", p[ 11 ] );
}
```

Backing up one element.

Missed this error.

On my system.

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
0
=================================================================
==509239==ERROR: AddressSanitizer: global-buffer-overflow on address ...
READ of size 4 at 0x55a7c7670108 thread T0
    #0 0x55a7c766d28e in function program.c:11
```

Use After Return

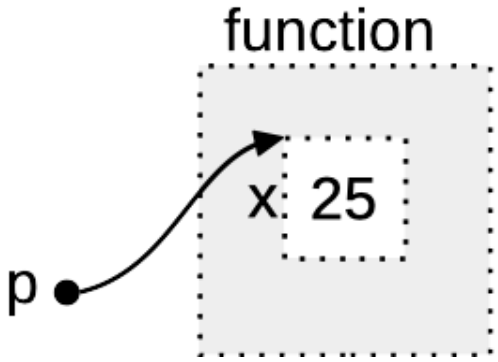
- Access a stack variable after it's gone?

```
int *function()
{
  int x = 25;
  return &x;
}
```

I got a compiler warning here.

```
  int *p = function();
  printf( "%d\n", *p );
```

The thing this points to is gone.



Use After Return

- Access a stack variable after it's gone?

```
int *function()
{
  int x = 25;
  return &x;
}
```

I got a compiler warning here.

The thing this points to is gone.

```
  int *p = function();
  printf( "%d\n", *p );
```

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
AddressSanitizer:DEADLYSIGNAL
=================================================================
==509394==ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000 …
==509394==The signal is caused by a READ memory access.
==509394==Hint: address points to the zero page.
    #0 0x565234da83ac in main program.c:14
```

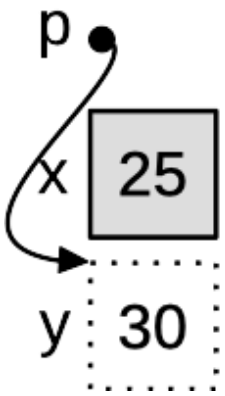
Use Out-of-Scope

- Access a stack variable after it's out of scope?

```
int x = 25;
int *p = &x;
{
  int y = 30;
  p = &y;
}
printf( "%d\n", *p );
```

No compiler warning here.

Technically, y goes away here.



Use Out-of-Scope

- Access a stack variable after it's out of scope?

```
int x = 25;
int *p = &x;
{
  int y = 30;
  p = &y;
}
printf( "%d\n", *p );
```

No compiler warning here.

Technically, y goes away here.

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
=================================================================
==509466==ERROR: AddressSanitizer: stack-use-after-scope on address ...
READ of size 4 at 0x7fff70a337f0 thread T0
    #0 0x55b5ed9d73b2 in function program.c:13
```

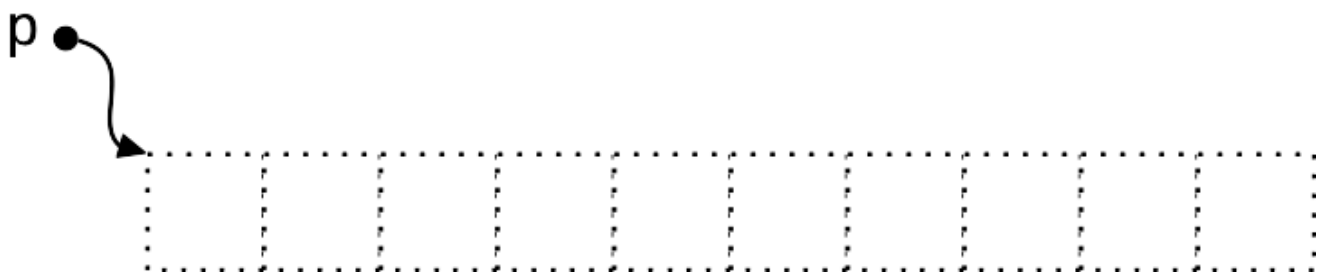
Use After Free

- Try to use a block of memory after you free it?

```
int *p = (int *) malloc( sizeof( int ) * 10 );
p[ 9 ] = 55;
free( p );

printf( "%d\n", p[ 9 ] );
```

You just freed this.



Use After Free

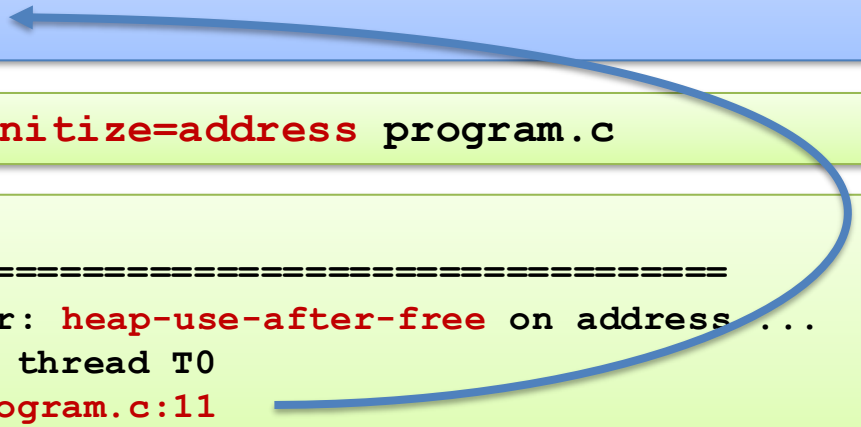
- Try to use a block of memory after you free it?

```
int *p = (int *) malloc( sizeof( int ) * 10 );
p[ 9 ] = 55;
free( p );

printf( "%d\n", p[ 9 ] );
```

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
=================================================================
==508862==ERROR: AddressSanitizer: heap-use-after-free on address ...
READ of size 4 at 0x604000000034 thread T0
    #0 0x561067c572f2 in main program.c:11
```



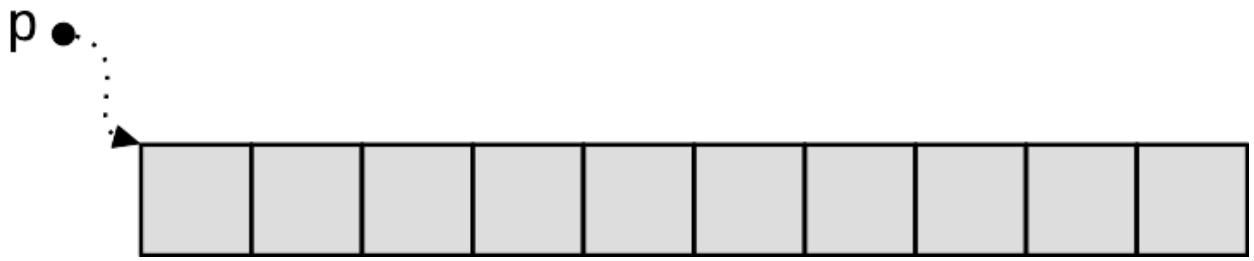
Memory Leak

- Dynamic memory leaks.

```
int main()
{
  int *a = (int *) malloc( sizeof( int ) * 10 );
  a[ 9 ] = 55;

  // Forget to free this block.
  return EXIT_SUCCESS;
}
```

You never free this.



Memory Leak

- Dynamic memory leaks.

```
int main()
{
  int *a = (int *) malloc( sizeof( int ) * 10 );
  a[ 9 ] = 55;

  // Forget to free this block.
  return EXIT_SUCCESS;
}
```

```
$ gcc -Wall -std=c99 -g -fsanitize=address program.c
```

```
$ ./program
=================================================================
==101739==ERROR: LeakSanitizer: detected memory leaks

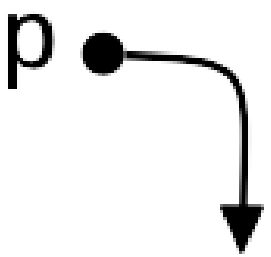
Direct leak of 40 byte(s) in 1 object(s) allocated from:
    #0 0x7fc6b9264b40 in __interceptor_malloc (..)
    #1 0x5589e90af84b in main program.c:7
    #2 0x7fc6b8db6bf6 in __libc_start_main (..)
```

Null Pointer Dereference

- Try to access memory via a NULL pointer?

```
int *p = NULL;
printf( "%d\n", *p );
printf( "%d\n", p[ 5 ] );
```

Can't dereference a NULL pointer.



Null Pointer Dereference

- Try to access memory via a NULL pointer?

```
int *p = NULL;
printf( "%d\n", *p );
printf( "%d\n", p[ 5 ] );
```

Can't dereference a NULL pointer.

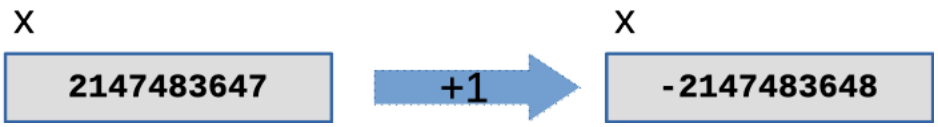
```
$ gcc -Wall -std=c99 -g -fsanitize=undefined program.c
```

```
$ ./program
program.c:8:3: runtime error: load of null pointer of type 'int'
Segmentation fault (core dumped)
```

Signed Integer Overflow

- The standard doesn't guarantee a particular behavior on signed overflow.
  - Although all modern systems exhibit the same behavior.

```
int x = INT_MAX;
x += 1;
printf( "%d\n", x );
```



Signed Integer Overflow

- The standard doesn't guarantee a particular behavior on signed overflow.
  - Although all modern systems exhibit the same behavior.

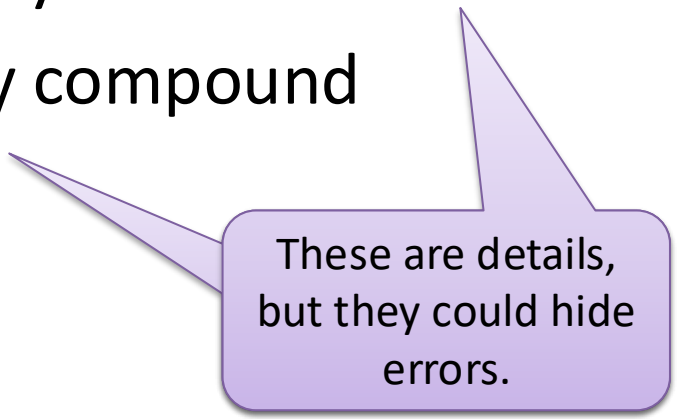
```
int x = INT_MAX;
x += 1;
printf( "%d\n", x );
```

```
$ gcc -Wall -std=c99 -g -fsanitize=undefined program.c
```

```
$ ./program
program.c:9:5: runtime error: signed integer overflow: 2147483647 + 1
cannot be represented in type 'int'
-2147483648
```

Code Coverage

- For white-box testing, we can see how much code gets executed under test.
- Ideally, all the source code should get a chance to run under test
  - Every statement executed
  - Every branch direction of every conditional
  - Every sub-expression of every compound conditional



Tools for Code Coverage

- The compiler will help us measure this
- Compile with the -fprofile-arcs and -ftest-coverage flags

```
gcc -Wall -std=c99 -fprofile-arcs \
    -ftest-coverage bubble.c -o bubble
```

- Run your program all you like

```
./bubble1 < small_input
./bubble1 < some_other_input
```

Using gcov

- Notice, you get some extra files.

```
eos$ ls
bubble              bubble.c
bubble.gcno         bubble.gcda
```

written during execution

written by the compiler

- Run gcov to extract the coverage information

```
eos$ gcov bubble.c
File 'bubble.c'
Lines executed:66.67% of 36
bubble.c:creating 'bubble.c.gcov'
```

Reading gcov Output

- In addition to the summary, gcov writes a text file giving coverage details.

```
     2:   42:  for ( int i=0; !done && i<len; i++ ) {
     -:   43:    // If we make it through without …
     1:   44:    done = true;
     -:   45:
     5:   46:    for ( int j=0; j<len-i-1; j++ )
     4:   47:      if ( list[j ] > list[j+1] ) {
 #####:   48:        swap( list + j, list + j + 1 );
 #####:   49:        done = false;
     -:   50:      }
     -:   51:  }
     1:   52:}
```

number of times executed … line number

Reading gcov Output

- gcov has some codes for reporting execution count

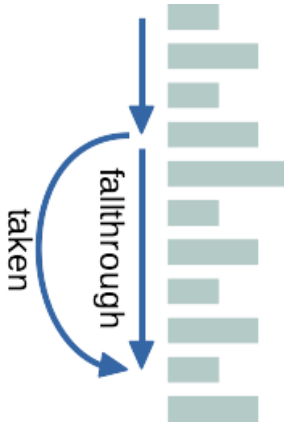
5	I was executed 5 times
-	I could never be executed (this doesn't count against coverage)
#####	I was never executed (but I could have been)

- Let's see if we can get code coverage up to 100 %

Getting More from gcov

- By default, gcov tells you about statements executed
- It can also tell you how branches went

```
eos$ gcov -b bubble.c
File 'bubble.c'
Lines executed:66.67% of 36
Branches executed:100.00% of 18
Taken at least once:77.78% of 18
Calls executed:61.54% of 13
bubble.c:creating 'bubble.c.gcov'
```



Reading gcov Output

- With the -b option, gcov reports branching directions for each part of each conditional.

```
      1:   40:  bool done = false;
      -:   41:
      2:   42:  for ( int i=0; !done && i<len; i++ ){
branch  0 taken 50% (fallthrough)
branch  1 taken 50%
branch  2 taken 100%
branch  3 taken 0% (fallthrough)
      -:   43:    // If we make it through …
      1:   44:    done = true;
```

Which way did **!done** go?

Which way did **i<len** go?

Meet Valgrind

Your new Best Friend

- Valgrind : a *dynamic-analysis* error detection tool
- It inserts error-checking instructions as it runs your code.
  - So, there can be considerable overhead, between 10x and 100x
  - Watching what our program does, and complaining about things it doesn't like
- Supports multiple tools, to check for different things.
  - We'll see two of them.

Meet Valgrind memcheck

- The default valgrind tool is memcheck
  - Command-line option: --tool=memcheck
  - Mostly looks for errors with heap-allocated memory
  - Out-of-bounds access to memory
  - Leaked memory
  - Command-line option: --leak-check=full
- Also looks for use of uninitialized static, stack or heap memory

But, this is the default anyway.

Using Valgrind

- Valgrind can tell you more if you include symbol information in your executable:

```
gcc -g -std=c99 –Wall program.c -o program
```

- In general, you run valgrind like this:

```
valgrind valgrind-options ./program program-options
```

- For example, you could check for memory errors and leaks with:

```
valgrind --tool=memcheck --leak-check=full ./program
```

Using Valgrind

- Bad code, using an uninitialized pointer.

```
int *x;
printf( "%d\n", *x );
```

I wonder where this points. Probably nowhere useful.

Looking at who-knows-where in memory.

Using Valgrind

- Bad code, using an uninitialized pointer.

```
int *x;
printf( "%d\n", *x );
```

```
$ valgrind ./program
==7264== Memcheck, a memory error detector
==7264== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==7264== Using Valgrind-3.19.0 and LibVEX; rerun with -h for copyright info
==7264== Command: ./test
==7264==
==7264== Use of uninitialised value of size 8
==7264==    at 0x4005A9: main (test.c:11)
==7264==
==7264== Invalid read of size 4
==7264==    at 0x4005A9: main (test.c:11)
==7264==  Address 0x0 is not stack'd, malloc'd or (recently) free'd
==7264==
  .
  .
  .
```

Using an uninitialized variable (a pointer).

Accessing a bad memory location.

A Bad Program

```
int staticArray[ 10 ];

int main()
{
  int stackArray[ 10 ];

  int *heapArray = (int *)malloc( 10 * sizeof( int ) );

  for ( int i = 0; i <= 10; i++ )
    heapArray[ i ] = 0;

  for ( int i = 0; i <= 10; i++ )
    staticArray[ i ] = 0;

  for ( int i = 0; i <= 10; i++ )
    stackArray[ i ] = 0;
```

Three regions of memory.

There's your problem right there.

Using Valgrind memcheck

- valgrind may generate a **lot** of output

```
$ valgrind --tool=memcheck --leak-check=full ./bounds
==4079== Memcheck, a memory error detector
==4079== Copyright (C) 2002-2015, and GNU GPL'd, by Julian Seward …
==4079== Using Valgrind-3.12.0 and LibVEX; rerun with -h for …
==4079== Command: ./bounds
==4079==
==4079== Invalid write of size 4
==4079==    at 0x4005B0: main (bounds.c:22)
==4079==  Address 0x51f9068 is 0 bytes after a block of size 40 …
==4079==    at 0x4C29BE3: malloc (vg_replace_malloc.c:299)
==4079==    by 0x40058E: main (bounds.c:17)
<more>
```

Here's what went wrong.

Here's where it went wrong.

Here's what you tried to access.

Here's where you allocated the block.

Meet Valgrind sgcheck

- Memcheck doesn't try to detect out-of-bounds access to stack or static arrays
- There's an experimental tool that tries to do that.
  - Command-line option: --tool=exp-sgcheck
  - It's for stack and global bounds checking
  - It applies some heuristic rules for detecting errors … it may miss some.
- You can enable sgcheck or memcheck, not both
  - But, you can run one then the other.

Using Valgrind sgcheck

```
$ valgrind --tool=exp-sgcheck ./bounds
==4925== … (greetings from valgrind) …
==4925== Command: ./bounds
==4925==
==4925== Invalid write of size 4
==4925==    at 0x4005CE: main (bounds.c:25)
==4925==  Address 0x601088 expected vs actual:
==4925==  Expected: global array "staticArray" of size 40 in object …
==4925==  Actual:   unknown
==4925==  Actual:   is 0 after Expected
==4925==
==4925== Invalid write of size 4
==4925==    at 0x4005F1: main (bounds.c:28)
==4925==  Address 0xffefffff08 expected vs actual:
==4925==  Expected: stack array "stackArray" of size 40 in this frame
==4925==  Actual:   unknown
==4925==  Actual:   is 0 after Expected
<a little more>
```

Static array out-of-bounds.

Stack array out-of-bounds.

Understanding Valgrind

- Understanding valgrind output will help you find bugs in your code.
  - Even bugs you don't see when you run the program.
- Valgrind produces lots of output
  - But, it's worth you time to look through to see if it finds any errors.
  - I'd start debugging from the first reported error.
  - … subsequent errors may depend on it.

Invalid Read/Write

- Valgrind will complain if you access memory beyond what you allocated.

```
int *list = (int *)malloc( 10 * sizeof( int ) );

int a = list[ 10 ];

list[ -1 ] =  a + 1;
```

```
==21537== Invalid read of size 4
==21537==    at 0x400522: main (invalid.c:13)
==21537==  Address 0x4c2706c is 0 bytes after a block of size 40 alloc'd
   ...
==21537== Invalid write of size 4
==21537==    at 0x400535: main (invalid.c:16)
==21537==  Address 0x4c2703c is 4 bytes before a block of size 40 alloc'd
```

Uninitialized Memory

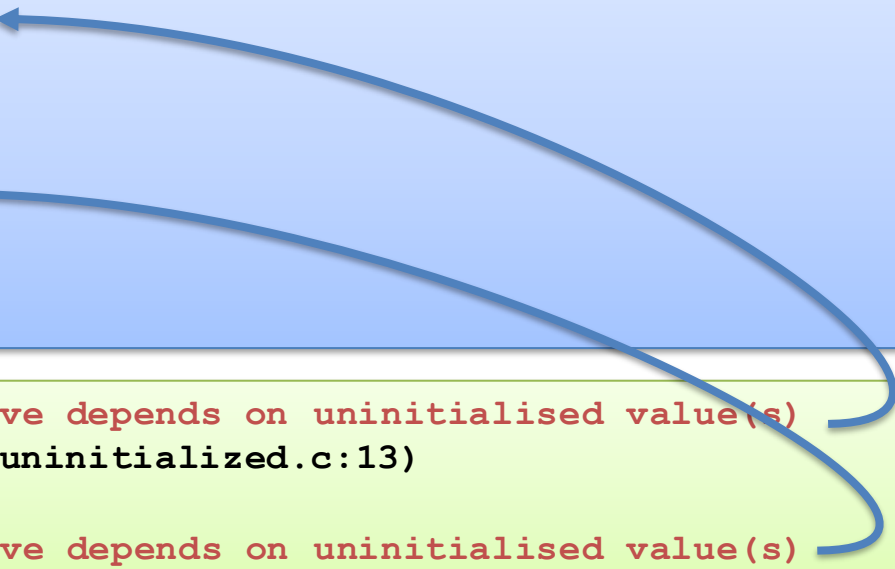
- It will notice if you make choices based on uninitialized memory.

```
int *list = (int *)malloc( 10 * sizeof( int ) );

if ( list[ 0 ] == 0 )
   ...;

int val = list[ 1 ];
if ( val )
   ...;
```

```
==3155== Conditional jump or move depends on uninitialised value(s)
==3155==    at 0x400572: main (uninitialized.c:13)
  ...
==3155== Conditional jump or move depends on uninitialised value(s)
==3155==    at 0x40058F: main (uninitialized.c:19)
```



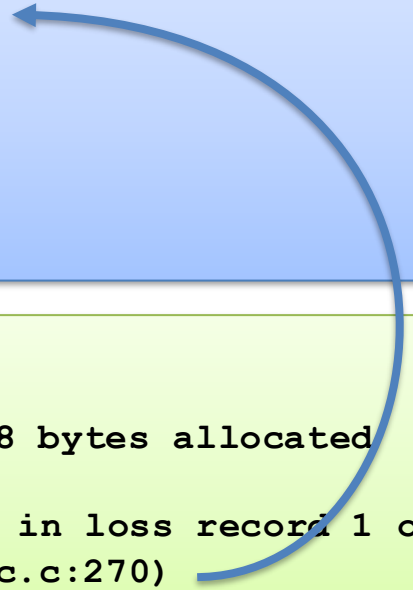
Leaks

- Valgrind will notice if you don't free your memory.

```
int main()
{
  char *buffer = (char *)malloc( 100 );
  FILE *fp = fopen( "abc.txt", "r" );

  return 0;
}
```

```
==3215== HEAP SUMMARY:
==3215==     in use at exit: 668 bytes in 2 blocks
==3215==   total heap usage: 2 allocs, 0 frees, 668 bytes allocated
==3215==
==3215== 100 bytes in 1 blocks are definitely lost in loss record 1 of 2
==3215==    at 0x4A06A2E: malloc (vg_replace_malloc.c:270)
==3215==    by 0x4005D5: main (leak.c:11)
```



File Leaks

- With the right options, valgrind will report file leaks also.

```
int main()
{
  char *buffer = (char *)malloc( 100 );
  FILE *fp = fopen( "abc.txt", "r" );

  return 0;
}
```

```
valgrind --track-fds=yes ./program
```

```
==3250== FILE DESCRIPTORS: 4 open at exit.
==3250== Open file descriptor 3: abc.txt
==3250==    at 0x37F08DB3B0: __open_nocancel (in /lib64/libc-2.12.so)
==3250==    by 0x37F0872AEE: _IO_file_fopen@@GLIBC_2.2.5 (in ...)
==3250==    by 0x37F0866F25: __fopen_internal (in /lib64/libc-2.12.so)
==3250==    by 0x4005EE: main (leak.c:12)
```

You get 3 streams automatically, so 4 or more is a leak

Static Analysis

- Valgrind is a dynamic analysis tool
  - It depends on running your program … and seeing if anything goes wrong.
- A static analysis tool depends on looking at your code
  - … and seeing if anything looks suspicious.
- We'll try out cppcheck

Remember Me?

```
int staticArray[ 10 ];

int main()
{
  int stackArray[ 10 ];

  int *heapArray = (int *)malloc( 10 * sizeof( int ) );

  for ( int i = 0; i <= 10; i++ )
    heapArray[ i ] = 0;

  for ( int i = 0; i <= 10; i++ )
    staticArray[ i ] = 0;

  for ( int i = 0; i <= 10; i++ )
    stackArray[ i ] = 0;
```

Three regions of memory.

Out-of-bounds errors.

Using cppcheck

- We run cppcheck on the source file

```
$ cppcheck bounds.c
Checking bounds.c...
[bounds.c:25]: (error) Array 'staticArray[10]' accessed at index 10,
which is out of bounds.
[bounds.c:28]: (error) Array 'stackArray[10]' accessed at index 10,
which is out of bounds.
[bounds.c:22]: (error) Array 'heapArray[10]' accessed at index 10,
which is out of bounds.
```

It found all three errors.

A Little Obfuscation

```
void zeroFill( int *a )
{
  for ( int i = 0; i <= 10; i++ )
    a[ i ] = 0;
}

int staticArray[ 10 ];

int main()
{
  int stackArray[ 10 ];

  int *heapArray = (int *)malloc( 10 * sizeof( int ) );

  zeroFill( heapArray );
  zeroFill( staticArray );
  zeroFill( stackArray );
```

The out-of-bounds access is now in this function.

Missing Errors with cppcheck

- Static analysis can't possibly detect every error

```
$ cppcheck bounds2.c
Checking bounds2.c...
```

Oops, didn't see the errors this time.

- It's good to have access to a combination of tools.
- In class, we'll compare our static- and execution-time tools.